ACES: Appliance Communications and Execution Standard

Version 0.2 – Updated Working Draft for Consortium Feedback
Date: 21/09/2026
Status: Open Draft
Maintained by: ACES Consortium

1. Scope

This document defines the ACES (Appliance Communications and Execution Standard), an open, vendor-neutral standard for physical and logical interoperability across modular robotics and IoT systems.

ACES defines a standardized module boundary through which devices can:

• Connect mechanically to a host system or other compatible infrastructure.
• Receive electrical power.
• Exchange data.
• Declare their capabilities and operational requirements.
• Expose functions that can be invoked using standardized execution semantics.

Work in parallel to provide the necessary energy and bandwidth
• Return status, errors, and telemetry.

ACES is intended to enable modular devices from different manufacturers, industries, and generations to be integrated without bespoke physical, electrical, or software integration.

The standard is industry-agnostic. Initial target domains include agriculture, smart cities, industrial automation, autonomous systems, sensing, energy, and field-deployed robotics.

ACES does not define system-level orchestration, optimization, routing, decision-making, or application logic. It also does not mandate a particular messaging protocol, network architecture, or transport technology.

Protocols such as MQTT, CAN, Ethernet, OPC UA, HTTP, ISOBUS, RF, cellular, and future transports may carry ACES-compatible execution semantics.

The objective of ACES is to standardize the boundary between software intent and physical capability while allowing both the hardware and software ecosystems on either side of that boundary to evolve independently.

2. Referenced and Related Standards

The following standards are relevant to ACES design and implementation.

Unless explicitly incorporated into a future ACES conformance profile, they are informative references rather than mandatory ACES dependencies.

• ISO/IEC 7498-1:1994 – Open Systems Interconnection Basic Reference Model.
• ISO 11898-1:2024 – Controller Area Network (CAN), data link layer and physical coding sublayer.
• IEC 61131-3:2025 – Programmable controllers, programming languages.
• ISO 11783 series – Serial control and communications data network for agricultural and forestry machinery (ISOBUS).
• OASIS MQTT Version 5.0 – Publish/subscribe messaging transport.
• JSON Schema Draft 2020-12 – Machine-readable data schema definition.
• IEEE 802.3 family – Ethernet.
• OPC UA Part 14, Version 1.05.06 – PubSub communication model.
• RFC 8446 – Transport Layer Security (TLS) 1.3.

ACES is intended to complement these technologies rather than replace them.

3. Definitions and Abbreviations

ACES Module

A physical device exposing one or more functions through an ACES-compliant module interface.

Examples include sensors, actuators, robotic arms, propulsion units, pumps, cameras, power systems, processing units, agricultural implements, and industrial equipment.

ACES Adapter

The standardized physical interface between an ACES module and the system to which it is connected.

An ACES adapter provides the required mechanical structure, electrical interface, and data transport.

The adapter is passive. It contains no orchestration logic, performs no system-level decision-making, and does not interpret the intended use of a module.

EDS – Electronic Data Sheet

A machine-readable description of an ACES module.

The EDS declares what the module can do, the parameters accepted by its functions, the telemetry it produces, its power requirements, interfaces, and relevant operational characteristics.

Capability

A function exposed by an ACES module that can be invoked by a compatible controlling system.

Controlling System

Any system capable of discovering ACES modules and invoking their capabilities.

A controlling system may be a PLC, edge computer, embedded controller, application, cloud service, AI system, human-operated system, or another ACES-compatible module.

Execution Contract

The standardized semantics governing how a capability is invoked and how acknowledgement, state, errors, completion, and telemetry are reported.

Transport Profile

A specification describing how ACES semantics are carried over a particular physical or communications technology.

Examples may include MQTT, CAN, Ethernet, OPC UA, or domain-specific transports.

4. Physical Interface Specification

4.1 Design Principles

ACES defines a standardized physical module interface covering:

• Mechanical coupling.
• Electrical power delivery.
• Data connectivity.
• Module presence and discovery mechanisms where applicable.
• Predictable scaling through multiple interfaces.

An ACES-compliant module may connect through one or more ACES adapters.

4.2 Passive Adapter Model

The ACES adapter shall act as a passive interface.

The adapter shall not:

• Perform orchestration.
• Determine how a module is used.
• Optimize system behavior.
• Contain application-specific decision logic.
• Require dependence on a specific software platform.

Its function is to provide a predictable physical interface through which power, data, and mechanical support can be delivered.

4.3 Horizontal Scaling

ACES is designed around horizontal scaling.

When a module requires more electrical power, data bandwidth, or mechanical support than one adapter can provide, multiple ACES adapters may be used in parallel.

This avoids requiring a single oversized connector capable of supporting every possible device class.

A low-power sensor may require only one interface.

A high-power actuator, propulsion unit, processing system, or industrial module may use several interfaces.

The module's EDS shall declare the interface resources it requires.

4.4 Interface Classes

ACES may define multiple physical interface classes appropriate to different operating environments and loads.

Each physical profile shall specify, where applicable:

- Mechanical dimensions and mating geometry.
- Alignment and retention mechanisms.
- Electrical voltage and current ranges.
- Grounding requirements.
- Electrical protection requirements.
- Data signaling interfaces.
- Environmental protection requirements.
- Mechanical load limits.
- Connection and disconnection behavior.
- Hot-plug or hot-swap requirements.
- Safety characteristics.

The exact connector geometry, electrical pin allocation, voltage classes, and signaling configuration are not fixed by this working draft and remain subject to consortium validation.

5. Logical Interface Specification

5.1 Transport Independence

The ACES core specification defines execution semantics independently of the underlying messaging or transport technology.

ACES does not require MQTT, CAN-FD, Ethernet, OPC UA, HTTP, ISOBUS, or any other specific transport.

A transport profile defines how ACES operations are represented on a particular transport.

This separation allows ACES-compatible modules to operate across different industries and communications environments without changing the underlying execution model.

5.2 Core Operations

An ACES implementation shall support the concepts required for the following operations:

DISCOVER

Determine that a compatible module is available.

DESCRIBE

Obtain the module's Electronic Data Sheet.

INVOKE

Request execution of a capability exposed by the module.

ACKNOWLEDGE

Confirm that an invocation has been received and indicate whether it has been accepted or rejected.

STATE

Report the current operational state of the module or an active invocation.

TELEMETRY

Return data generated by the module.

ERROR

Report an error or abnormal condition using standardized error semantics.

CONFIGURE

Modify permitted module parameters where configuration is supported.

5.3 Electronic Data Sheet

Each ACES module shall expose an Electronic Data Sheet.

The EDS provides a machine-readable description of the module and shall contain sufficient information for a controlling system to determine:

• Module identity.
• Manufacturer.
• Module type and version.
• Supported ACES specification version.
• Available capabilities.
• Parameters accepted by each capability.
• Data or result structures returned by those capabilities.
• Available telemetry.
• Power requirements.
• Physical interface requirements.
• Operational constraints relevant to safe invocation.

The EDS describes what a module can do.

It does not define when, why, or in what sequence those capabilities should be used.

5.4 Example Electronic Data Sheet

The following JSON representation is illustrative. JSON is not mandated as the only possible serialization format.


```
{
"aces_version": "0.2",
"module_id": "pump-12A3",
"manufacturer": "Irritech",
"model": "FlowPump-120",
"module_version": "1.2.0",

"capabilities": [
{
"name": "start"
},
{
"name": "stop"
},
{
"name": "set_flow",
"parameters": {
"value": {
```

```
"type": "number",
"unit": "L/min"
}
}
}
],

"telemetry": {
"flow_rate": {
"type": "number",
"unit": "L/min"
},
"pressure": {
"type": "number",
"unit": "kPa"
}
},

"power_profile": {
"nominal_w": 120,
"peak_w": 300
},

"interfaces": {
"required_adapters": 1
}
}
```


5.5 Function Invocation

A controlling system invokes a function declared in the module’s EDS.

An invocation shall include sufficient information to identify:

- The target module.
- The requested capability.
- Input parameters.
- A unique invocation identifier.
- Relevant timing or validity information where applicable.

Illustrative representation:


```
{
"invocation_id": "job-804921",
"module_id": "pump-12A3",
"function": "set_flow",
```

"parameters": {
"value": 12.5
},
"timestamp": "2026-09-21T07:45:00Z"
}

5.6 Acknowledgement and Execution State

A module shall provide an acknowledgement for an invocation.

The acknowledgement shall indicate whether the invocation was:

• Accepted.
• Rejected.
• Invalid.
• Unsupported.
• Unable to execute because of the module’s current state.

Long-running operations should expose their execution state.

Typical execution states may include:

• accepted
• queued
• running
• completed
• failed
• cancelled

5.7 Errors

Error reporting shall distinguish between:

• Invalid requests.
• Unsupported capabilities.
• Invalid parameters.
• Resource limitations.
• Safety constraints.
• Hardware faults.
• Communication faults.
• Execution failures.

Transport profiles may define how these error states are encoded, but their logical meaning shall remain consistent.

5.8 Telemetry

ACES defines how module telemetry is described and exposed at the execution boundary.

ACES does not define:

• Where telemetry is ultimately stored.
• How telemetry is aggregated.
• How telemetry is analyzed.
• Which database or cloud platform receives it.
• How routing is performed outside the ACES module boundary.

These responsibilities remain with higher-level systems.

5.9 Module Lifecycle

A typical ACES module lifecycle is:

1. Module is physically connected.
2. The system detects the module.
3. The module exposes its EDS.
4. The controlling system reads and validates the EDS.
5. The module's available capabilities become accessible to the controlling system.
6. Capabilities are invoked as required.
7. Acknowledgements, state, errors, and telemetry are returned.
8. If the module is removed or replaced, the system updates its representation of available capabilities.

A replacement module may expose a different implementation while maintaining compatible ACES capabilities.

6. Security

6.1 Scope

ACES defines security requirements at the module execution boundary but does not mandate a single security architecture for every transport or deployment.

Security mechanisms shall be appropriate to the selected ACES transport profile and deployment environment.

6.2 Security Objectives

ACES deployments should provide mechanisms for:

• Module identification.
• Authentication where required.
• Authorization of function invocation.
• Message integrity.

• Protection against unauthorized or malformed commands.
• Replay protection where relevant.
• Confidentiality where operational data requires it.
• Traceability of security-relevant operations where required.

Transport-specific security shall be defined by the applicable profile.

For example, an IP-based MQTT profile may use TLS 1.3 and mutual authentication, while a CAN-based profile may use mechanisms appropriate to the capabilities and constraints of that network.

6.3 Safety and Security Separation

Security authorization does not override module-level safety constraints.

A correctly authenticated command may still be rejected if execution would violate a module's declared safety or operational limits.

7. Conformance and Certification

7.1 Core Conformance

ACES core conformance shall test:

• EDS validity.
• Capability declaration.
• Function invocation semantics.
• Acknowledgement behavior.
• State reporting.
• Error reporting.
• Telemetry description.
• Version compatibility.

7.2 Physical Conformance

Physical conformance testing shall be profile-specific and may include:

• Connector mating and retention.
• Electrical compatibility.
• Mechanical tolerances.
• Environmental requirements.
• Power delivery.
• Data integrity.
• Multiple-adapter operation.
• Connection and disconnection behavior.

7.3 Transport Profile Conformance

An implementation claiming compatibility with a specific ACES transport profile shall pass the tests defined for that profile.

A module may support more than one ACES transport profile.

7.4 Certification

The ACES Consortium intends to maintain a formal conformance and certification framework.

The framework is expected to include:

- Core ACES compliance criteria.
- Physical interface profile tests.
- Transport profile tests.
- EDS validation tooling.
- Reference implementations.
- Certification requirements and levels.

The formal certification program is not yet normative in this working draft.

8. Versioning

ACES specifications shall use semantic versioning.

MAJOR

Breaking changes to the ACES module boundary, physical compatibility requirements, EDS contract, or execution semantics.

MINOR

Backward-compatible capabilities, fields, profiles, or execution semantics.

PATCH

Clarifications, corrections, editorial changes, and other non-breaking updates.

Each EDS shall identify the ACES specification version against which it was implemented.

Individual physical and transport profiles may maintain their own version numbers independently of the ACES core version.

9. Licensing and Governance

ACES is intended to be an open, vendor-neutral, royalty-free standard.

The ACES core specification and associated reference material shall be released under a permissive licensing model that permits implementation in both open-source and proprietary products.

The final software, specification, contribution, and intellectual-property licensing framework shall be defined by the ACES Consortium as part of its governance process.

No ACES-compliant implementation shall be required to use a specific commercial platform, software stack, manufacturer, or consortium member.

Commercial value is expected to be created independently by participating manufacturers, integrators, software providers, and service providers using the standard.

10. Separation of Responsibilities

ACES Standardizes

- Physical plug-and-play module interfaces.
- Capability declaration through the EDS.
- Standardized function invocation.
- Acknowledgements.
- Execution state.
- Errors.
- Telemetry semantics.
- Compatibility and conformance requirements.

ACES Does Not Standardize

- System-level orchestration.
- Decision-making.
- Optimization.
- Sequencing.
- Operational policy.
- Application-specific behavior.
- Data storage architecture.
- Analytics.
- AI models.
- Network routing.
- A mandatory messaging protocol or transport technology.

This separation of responsibilities is fundamental to the ACES architecture.

11. Annexes

Annex A: Preliminary EDS Schema

The following schema represents the current working direction for the ACES Electronic Data Sheet.

It is not yet a final normative schema.

```
{
"$schema": "https://json-schema.org/draft/2020-12/schema",
"$id": "https://aces-iot.org/schema/eds/0.2",
"title": "ACES Electronic Data Sheet",
"type": "object",

"required": [
"aces_version",
"module_id",
"manufacturer",
"model",
"module_version",
"capabilities",
"power_profile"
],

"properties": {

"aces_version": {
  "type": "string"
},

"module_id": {
  "type": "string"
},

"manufacturer": {
  "type": "string"
},

"model": {
  "type": "string"
},

"module_version": {
  "type": "string"
},

"capabilities": {
  "type": "array",
```

```
  "items": {
    "type": "object",
    "required": ["name"],
    "properties": {
      "name": {
        "type": "string"
      },
      "parameters": {
        "type": "object"
      },
      "returns": {
        "type": "object"
      }
    }
  }
},

"telemetry": {
  "type": "object"
},

"power_profile": {
  "type": "object",
  "required": [
    "nominal_w",
    "peak_w"
  ],
  "properties": {
    "nominal_w": {
      "type": "number",
      "minimum": 0
    },
    "peak_w": {
      "type": "number",
      "minimum": 0
    }
  }
},

"interfaces": {
  "type": "object"
},

"constraints": {
```

```
  "type": "object"
}


}
}
```

Annex B: Physical Interface Definition

The original ACES 0.1 working draft proposed a fixed 8-pin, 48 V DC interface.

The current ACES architecture instead treats the final physical interface as a consortium design decision requiring cross-domain validation.

The physical specification shall ultimately define:

- Connector geometry.
- Pin or contact definitions.
- Power classes.
- Data interfaces.
- Grounding and shielding.
- Mechanical support.
- Environmental protection.
- Hot-plug behavior.
- Multi-adapter alignment and aggregation.
- Safety mechanisms.

Until these parameters are validated and approved by the consortium, no specific connector, pinout, voltage, or current rating should be considered normative ACES.

Annex C: Example MQTT Transport Profile

This annex is non-normative.

MQTT Version 5.0 may be used as one transport for ACES semantics.

An MQTT binding could, for example, expose channels corresponding to:

manifest
invoke
ack
state
telemetry
config
error

An illustrative topic structure might be:

aces/<system_id>/<module_id>/

Example:

aces/farm-01/pump-12A3/invoke

The exact MQTT topic hierarchy, broker architecture, QoS levels, authentication mechanism, retained-message behavior, and payload serialization shall be specified by the MQTT transport profile rather than the ACES core specification.

Annex D: Example CAN Transport Profile

This annex is non-normative.

CAN or CAN-FD may be used to transport ACES execution semantics in systems where those technologies are appropriate.

A CAN profile would define deterministic mappings for:

- Module identification.
- EDS discovery or retrieval.
- Capability invocation.
- Acknowledgements.
- State.
- Errors.
- Telemetry.

The ACES core specification does not allocate fixed CAN identifier ranges.

Such allocation belongs to the applicable CAN transport profile.

Annex E: Preliminary Certification Test Categories

Physical Tests

- Connector compatibility.
- Mechanical mating and retention.
- Electrical compatibility.
- Data interface integrity.
- Environmental profile compliance.
- Multiple-adapter operation where supported.

EDS Tests

• Schema validation.
• Required identity fields.
• Capability declaration.
• Parameter definitions.
• Telemetry definitions.
• Power and interface requirements.

Execution Tests

• Capability discovery.
• Function invocation.
• Positive acknowledgement.
• Rejected invocation behavior.
• Execution state reporting.
• Error reporting.
• Telemetry generation.

Interoperability Tests

• Module substitution.
• Modules from different manufacturers.
• Multiple transport implementations where claimed.
• Addition and removal of modules without bespoke system integration.

Security Tests

Security tests shall be defined by the applicable transport and deployment profile.

Annex F: Architectural Principle

ACES occupies the boundary between software intent and physical action.

Higher-level systems determine what should happen.

Messaging and transport technologies carry information through the system.

ACES defines how a physical module declares its capabilities, receives a function invocation, executes it, and reports the result.

This boundary enables physical modularity without requiring a shared orchestration platform, common vendor, or fixed communications architecture.

Contact

ACES Consortium Coordination Unit
info@aces-consortium.org

https://www.aces-consortium.org